

DOMAIN-SPECIFIC ARCHITECTURE

Computer architecture is attributes of a system visible to the programmer which has direct impact on execution of the program (Hennessy & Patterson, 2018). There are various types of computer architecture such as Von-Neumann Architecture, Harvard Architecture, RISC (Reduced Instruction Set Computing) & CISC (Complex Instruction Set Computing) Architecture, and Domain-Specific Architecture (DSA) which is a programmed architecture specifically optimized for a certain domain (Wikipedia contributors, (2025)). The need for DSAs started with reaching the limits of general purpose architecture, in 1965 Gordon Moore made a significant prediction which is called Moore's Law (A prediction says that transistors on single chip can be doubled each year) (G. E. Moore, 1965) and in 1974, Dennard Scaling introduced and showed that we can build smaller transistors with constant power density because voltage and current will decrease at the same time, but through time Dennard Scaling started to fail due to transistors became so small that voltage stopped dropping and transistors started to consume power and overheat which showed that there was a need for another perspective which is Domain- Specific Architecture. (Mark Bohr, 2007; Esmailzadeh et al., 2011) DSA is optimized for certain tasks and functions rather than general purpose architectures which causes the ability to perform a specific task faster, consuming less energy, making the efficiency of execution better. (Hameed et al., 2010) Rather than putting in an abundant number of small transistors which started to be inconvenient to do all tasks, we choose to build architecture for specific domains, making the process better even though DSAs has some disadvantages like high costs cause they are hard to build and design , not being flexible(if an algorithm or domain changes than the DSA becomes useless) (Kumar & Hinton, 2004; Hennessy & Patterson, 2018) ,we know that DSAs made enormous positive change on technologies which is the reason that we choose to do DSA .We understand the fact that DSA is widely used, it has a huge impact on a lot of significant domains such as Artificial Intelligence (AI), image and video processing, cryptography and security, gaming and graphics, data center accelerators. (Jouppi et al., 2018) Another reason we choose DSAs is to understand the history of architecture, seeing the processes of designing and building different architectures, how they became efficient and the approach lays under architecture.

In this report, we will obtain information about:

- . The history of DSAs elaborately
- . The difference between general purpose architecture and specific domain architecture
- . The advantages of DSAs such as higher performance, which come from that it is optimized to work on certain tasks, lower energy consumption, higher efficiency.
- . The disadvantages like flexibility issues, high cost to design and build because it needs more engineering ability and time, and the possibility of being inconvenient in the future
- . The first usage examples of DSAs, the evolution of DSAs and how FPGA (Field Programmable Gate Array which enables reprogramming) and ASIC (Application -Specific Integrated Circuit which is a circuit designed for specific applications or function) affected DSAs . Different examples of DSAs like GPU (Graphics Processing Units), TPU (Tensor Processing Unit), NPU (Neural Processing Unit microprocessor designed to copy the process of human brain). We will give our recommendations for improvement.

In the end, we will give a brief information highlighting DSAs importance is to make process faster focusing on certain application, with the help of Moore's Law and Dennard Scaling this approach raised and today is used on AI, image processing and other domains and there are FPGA and ASIC

helped the improvement of DSAs so we see different types of DSAs like TPU, GPU, NPU which are used widely.

Domain-Specific Architecture (DSA) has recently sparked extensive discussions in the field of computer architecture, as opposed to the conventional one-size-fits-all approach, the designs are targeted towards certain application domains. Before we delve into DSA's significance, it is important to briefly look at the evolution of computer architecture as conventionally developed and understand the challenges that have been faced with the use of general-purpose designs.

CPUs for general purposes ever since the dawn of computing. These are general-purpose CPUs that, while not being great at anything, can do many things okay (*Hennessy & Patterson, 2019*). Their generality allowed them to be flexible, but it also meant that they were inefficient in dealing with very specialized workloads. For example, workloads such as rendering, scientific computing, machine learning could not effectively be done by legacy CPUs. evidently, became the inefficiency of the general-purpose processors.

Shortcomings of general-purpose architecture led to the evolution of Domain-Specific Architectures. DSAs are HW platforms specialized for particular workloads, sacrifice general applicability for performance and energy efficiency (*Esmailzadeh et al., 2011*). The result is a system that is far faster than general-purpose CPUs for targeted applications. DSAs have already become a gamechanger, enabling tailored solutions that are not achievable with current architectures.

An early and most popular instance of DSA is the Graphics Processing Units. Conceived first as a means to speed up image rendering and graphics operations, the GPUs were specialized machines for performing parallel computations in an efficient manner. During the late 90's, early innovation related to accelerating use of the GPU emerged from both NVIDIA and ATI Technologies (*Owens et al., 2008*). While tightly-focused on parallelism, GPUs showed how dedicated hardware could outperform general-purpose CPUs in a narrow application space leading to a more general acceptance of DSA.

Following GPUs, the need for even more domain-optimized performance led to the development of Field-Programmable Gate Arrays (FPGAs) and Application-Specific Integrated Circuits (ASICs). FPGAs allow for the reconfiguration of hardware post-manufacture at the expense of flexibility for even higher performance relative to CPUs (*Kuon & Rose, 2007*). This flexibility helped to render FPGAs extremely attractive to industries that needed custom hardware solutions without paying the high cost of manufacturing custom chips. On the other hand, ASICs are tailored to a single task or application and provide maximum performance at the cost of flexibility. ASICs are typically used where performance and power consumption are important, e.g., mobile phones and embedded systems.

The growth of machine learning and artificial intelligence has also facilitated the uptake and evolution of DSAs. Google created Tensor Processing Units (TPUs) expressly to accelerate machine learning workloads, with unimagined performance using conventional CPU architectures (*Jouppi et al., 2017*). Likewise, Neural Processing Units (NPUs) have been created to provide power-efficient AI inference, especially in mobile and edge contexts. These innovations point towards the direction of even more specific hardware that will be capable of addressing the specific needs of contemporary computational pursuits.

In summary, the evolution from conventional general-purpose CPUs to highly specialized DSAs indicates the increasing need for custom-fit solutions in computing. As applications increasingly become more complex and demanding, DSAs offer a way forward to meeting the performance and energy efficiency specifications. This historic transition is the key to understanding modern trends and

development in computer architecture today. As technology keeps evolving, DSAs are likely to become even more central to the future of computing, especially in the fields of artificial intelligence, Internet of Things (IoT), and real-time data processing.

DSAs are architectures which are really good at doing something in a specific area. On the other side general purpose processors can do lots of things but since it is not specialized it is not very efficient for all tasks. Because of this DSAs are faster and more efficient. According to Hennessy and Patterson (2019), the first DSAs were invented to solve these problems. They are faster and use less energy in their target domain (*Hennessy & Patterson, 2019*).

One of the biggest advantages of DSA's is that they can provide really high-performance gains. As an example, we can point out the TPU. Google's Tensor Processing Unit (TPU) is a application specific hardware. It works much more efficiently and faster than our regular CPUs and GPUs. *Jouppi et al. (2017)* have shown that the TPUs built to speed up the machine learning operations. Also, *Sze et al. (2017)* emphasized that deep neural networks require hardware solutions for these specific tasks. DSAs optimize the hardware for better performance in operations.

Energy efficiency is another important benefit of DSAs, especially in applications where high performance is needed. General-purpose processors often waste lots of energy on tasks because they are not optimized for that task. But when we look at the DSAs, since they are designed for specific areas, they minimize unnecessary operations, and they provide high performance without consumption of power. *Hameed et al. (2010)* explained that general-purpose processors are not very efficient because they have problems with handling instructions and using their pipelines well, so this causes them to use more energy. This is very important for places like data centers, mobile devices, and other systems where saving energy matters a lot.

There are also limitations for DSAs. It is harder to design and program them. Normal processors like CPUs and GPUs have many ready-to-use tools and it is easier to program and design. But DSAs often need special tools and programming methods. For example, *Putnam and his team (2014)* talked about using FPGAs in data centers. And it showed that there are not enough tools to do this. Also, regarding the design of the DSAs we have options like making the hardware flexible or making it work in a specialized area.

Besides all the benefits they can also have some security problems. They can be easier to attack. They might not have strong security like regular processors. *Hennessy and Patterson (2019)* say that if security is not carefully planned during the design, DSAs can have weak points which attackers might use.

DSAs are used in many different areas, such as machine learning, signal processing, and cryptography. Although they provide many benefits during their use, they have several potential vulnerabilities that can be encountered in long-term use. Examples of these vulnerabilities include lack of flexibility, security risks, manufacturing costs, and updateability.

One of the biggest potential vulnerabilities for DSAs is **lack of flexibility**. First of all, DSAs are structures optimized for a specific area. These structures only take on one task and choose to complete it as their goal. While this provides efficiency for machine learning, it makes DSAs inefficient for multi-purpose use. (*Kumar, R., & Hinton, G. (2004)*). Secondly, DSAs are "hard-coded", meaning they cannot be changed, and are mostly used for video processing. Since there is no flexibility in the system, this vulnerability causes efficiency to decrease. (*Wikipedia contributors (2025)*). Thirdly, the software in the hardware of DSAs is designed to work only with that architecture, so the software may have difficulty running on a different platform. (*Wikipedia contributors (2025)*). Finally, the lack of

flexibility in DSAs also brings with it a cost problem. While DSAs made in specific areas can be efficient in their own areas, they cannot be used in different areas, and this requires new architecture, which increases the cost. (Wikipedia contributors. (2025)).

Another potential vulnerability for DSAs is **security vulnerabilities**. Security vulnerabilities are important vulnerabilities that can cause unauthorized people to enter the system and damage or destroy data. DSA systems have accelerators and customized chips, but since security experience is not done on these structures, they can be exposed to low-level cyber-attacks such as side-channel attacks. (Shen, X., & Wang, Y. (2021)). At the same time, since DSAs are designed for high performance, sufficient security checks cannot be made during the optimization process, and therefore, a security vulnerability occurs on low API or firmware. (Shen, X., & Wang, Y. (2021)). Another reason for security vulnerabilities is the lack of isolation in DSAs, which allows one user to access another user's data. This situation poses a security risk, especially in the cloudy environment. (Johnson, T., & Singh, R. (2019)).

Another potential vulnerability for DSA is the **manufacturing cost**. DSAs are usually made with a special chip called an ASIC (Application-Specific Integrated Circuit) design. ASIC design is expensive and involves many steps. (Hennessy, J.L., & Patterson, D.A. (2018)). Today, making a 7 nm ASIC design costs about 100 million USD in R&D (Semico Research Corporation (2020)). In addition, mask set production, that is, the re-creation of photomask sets used in the production of the chip increases the cost. A 5 nm mask set costs about 20 million USD. (McKinsey & Company (2021)).

The last potential vulnerability for DSAs is their **updatability problems**. First of all, DSAs also operate with fixed hardware logic. In other words, while small changes can be made to the software, changes to the hardware are not allowed. (Hedge, R., & Keckler, S.W. (2017)). Additionally, since DSAs have a fixed structure, they may have difficulty responding to rapidly changing algorithms. Although this provides benefits in the fields of machine learning and image processing, it causes vulnerabilities in other areas. (Jouppi, N.P., & Google Research Team (2017)). Furthermore, while changes can be made to some simple parts of the firmware (parameters, etc.) updates cannot be made to the hardware and remanufacturing is required, and this production is very expensive in terms of costs. (Batten, C., & Fowers, J. (2019)).

Domain-Specific Architectures (DSAs) offer high efficiency and performance for specific application areas, while they also lead to various vulnerabilities due to their unique structure. Good security provides confidentiality, integrity and availability assurances against attacks and misuse of your data and systems.

Vulnerability Reduction Strategies:

Network Segmentation

Network segmentation is a security approach that divides a corporate network into smaller areas, allowing easier control of each area. In this way, malware cannot easily access one network from another. This makes access to other regions difficult and isolated. In this way, by dividing them into zones, they are more protected and separated from each other (Reichenberg, Nimmy (2014)).

Access Controls

With effective access control, only authorized users are allowed to access the system and data. In this way, the number of people who can only access and know is reduced to a minimum and can be controlled. There are two types of control mechanisms.

1. **Two-factor authentication (2FA)**, verification must be made not only by using a password but also by a second verification. (*Maternal Virtual Solution GmbH. (2019)*)
2. **Role-based access control (RBAC)**, allows the user to access only the resources in the job description, thus minimizing possible risks. (*Miller, D. (2025)*)

Encryption

Encrypting private data makes it difficult to access data and makes it difficult to read even if it is accessed. Using passwords during both data transfer and storage helps to maintain data security. Using encryption ensures the protection of customer information, payment data and personal data. In this way, it is ensured that the occurrence of company security vulnerabilities is prevented.

Patch Management

Security vulnerabilities in software systems are resolved with patches released by manufacturers. Regular release of these updates by companies is one of the most effective ways to close attacks and security gaps. In this way, security gaps that are found or overlooked are improved with updates made by companies. (*Cybersecurity and Infrastructure Security Agency. (2023.)*)

Vulnerability Scanning

By using automatic vulnerability scanners, weak points in the system can be understood in advance. This can be determined with scans to be performed in advance. With these scans, vulnerabilities arising from existing risks can be detected in advance and intervened early. In this way, it is possible to be protected against cyber-attacks. (*Cyflare. (2023.)*)

Security Awareness Training

The weakest point of the security chain is the human factor. Employees can often be deceived by methods such as sending fake e-mails and messages. For this reason, employees need to be given cyber security (*Alert Logic, 2018.*)

DSA architecture offers advantageous structures in terms of performance, but due to their uniqueness, they also bring new security threats. In such architectures, not only general security strategies but also specific approaches should be used. Security should be considered as a process that covers all features and possible situations, not just a feature in these systems. The process of correcting security vulnerabilities is of great importance for every company. Finding and correcting security vulnerabilities creates a high line of defense against cyber-attacks. Correcting security vulnerabilities not only provides financial protection but also helps with reputation and legal compliance.

In conclusion, DSA is a programmed architecture which is optimized to work on specific domains making the process more efficient. DSA has evolved from general-purpose CPUs to specialized hardware like GPUs, FPGAs, and TPUs. These architectures offer better performance and efficiency for specific tasks. The studies that we see about Domain-Specific Architectures offer significant performance and energy benefits for specialized tasks. However, careful design, especially in terms of security and usability is essential to fully realize their potential. DSAs have potential vulnerabilities due to lack of flexibility, security vulnerabilities, high production costs, and updateability issues. These vulnerabilities can cause problems in the long-term use of DSAs. To reduce security vulnerabilities, regular security updates, cybersecurity training for employees, and tools such as encryption, access controls, and vulnerability scanning must be used.

In the future, DSAs may take shape with more flexible hardware and security solutions to meet the needs of AI, IoT, and other technologies. This architecture will provide benefits and convenience in terms of speed and efficiency.

REFERENCES

1. Esmaeilzadeh, H., Blem, E., St. Amant, R., Sankaralingam, K., & Burger, D. (2011). Proceedings of the 38th Annual International Symposium on Computer Architecture, 365-376.
2. Hennessy, J. L., & Patterson, D. A. (2018). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.
3. Jouppi, N. P., Young, C., Patil, N., et al. (2017). "In-Datcenter Performance Analysis Proceedings of the 44th Annual International Symposium on Computer Architecture, 1-12.
4. Kuon, I., & Rose, J. (2007). "Measuring the Gap Between FPGAs and ASICs." IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 26(2), 203-215.
5. Owens, J. D., Houston, M., Luebke, D., Green, S., Stone, J. E., & Phillips, J. C. (2008). "GPU Computing." Proceedings of the IEEE, 96(5), 879-899.
6. Hennessy, J. L., & Patterson, D. A. (2019). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.
7. Jouppi, N. P., et al. (2017). In-datcenter performance analysis of a Tensor Processing Unit. ISCA 2017.
8. Sze, V., et al. (2017). Efficient processing of deep neural networks: A tutorial and survey. Proceedings of the IEEE, 105(12), 2295–2329.
9. Hameed, R., et al. (2010). Understanding sources of inefficiency in general-purpose chips. ACM SIGARCH Computer Architecture News, 38(3), 37–47.
10. Putnam, A., et al. (2014). A reconfigurable fabric for accelerating large-scale datacenter services. ISCA 2014.
11. Hennessy, J. L., & Patterson, D. A. (2019). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.
12. Kumar, R., & Hinton, G. (2004). Domain-Specific Architectures: - IEEE Transactions on Computers.
13. Wikipedia contributors. (n.d.). Domain-specific architecture. Wikipedia. Retrieved April 29, 2025, from https://en.wikipedia.org/wiki/Domain-specific_architecture
14. Shen, X., & Wang, Y. (2021). Security Implications of Domain-Specific Accelerators. IEEE Micro, 41(5), 72–79.
15. Johnson, T., & Singh, R. (2019). Isolation in Accelerators: Security Challenges in Multi-Tenant Systems. Proceedings of HotCloud '19.
16. Hennessy, J. L., & Patterson, D. A. (2018). Domain-Specific Architectures: IEEE Micro, 38(6), 36–45. Google Research Blog, The Tensor Processing Unit: An Overview, 2017.
17. Semico Research Corporation. (2020). ASIC Design Costs. Semico Research.
18. McKinsey & Company. (2021). The Economics of Semiconductor Manufacturing. McKinsey & Company Report.
19. Hegde, R., & Keckler, S. W. (2017). Limits of Flexibility in Domain-Specific Accelerators. ACM Transactions on Architecture and Code Optimization (TACO), 14(4), 1–25.
20. Jouppi, N. P., & Google Research Team. (2017). The Tensor Processing Unit: An Overview. Google Research Blog.
21. Batten, C., & Fowers, J. (2019). Flexibility vs. Efficiency in Hardware Accelerators. IEEE Proceedings, 107(1), 50–67.
22. Reichenberg, N. (2014, Oct 30). SecurityWeek <https://www.securityweek.com/understanding-it-risk-business-perspective/>

23. Scytale. (n.d.). Vulnerability mitigation. Scytale. Retrieved May 3, 2025, from <https://scytale.ai/glossary/vulnerability-mitigation/>
24. Sternum. (n.d.). Operating system vulnerabilities: Retrieved May 3, 2025, from <https://sternumiot.com/iot-blog/operating-system-vulnerabilities-understanding-and-mitigating-the-risk/>
25. Virtual Solution. (n.d.). Zwei-Faktor-Authentifizierung. Retrieved May 3, 2025, from <https://www.virtual-solution.com/glossar/zwei-faktor-authentifizierung>
26. Cybersecurity and Infrastructure Security Agency (CISA). (n.d.). Understanding patches and software updates. Retrieved May 3, 2025,
27. Alert Logic. (n.d.). *Why humans are the weakest link in cybersecurity.* <https://www.alertlogic.com/blog/why-humans-weakest-link-cybersecurity>
28. Miller, D. (2025, Şubat 4). *What is Role-Based Access Control (RBAC)?*. Varonis. <https://www.varonis.com/blog/role-based-access-control>
29. Mark Bohr, "A 30 Year Retrospective on Dennard's MOSFET Scaling Paper," IEEE Solid-State Circuits Society Newsletter, vol. 12, no. 1, pp. 11–13, Winter 2007.
30. G. E. Moore, "Cramming more components onto integrated circuits," Electronics, vol. 38, no. 8, pp. 114–117, Apr. 1965
31. Chi, Y., Qiao, W., Sohrabizadeh, A., Wang, J., & Cong, J. (2022). Democratizing domain-specific computing. arXiv preprint arXiv:2209.02951. <https://arxiv.org/abs/2209.02951>
32. Hameed, R., Yehia, S., & Mudge, T. (2010). Domain-specific architectures: ACM Transactions on Architecture and Code Optimization, 19(4), 1–23.
33. Jouppi, N. P., Young, C., Patil, N., & Patterson, D. (2018). A domain-specific architecture for deep neural networks. Communications of the ACM, 61(9), 50–59.
34. Kumar, R., & Hinton, G. (2004). A family of 45nm IA processors. IEEE International Solid-State Circuits Conference Digest of Technical Papers, 58–59.